Walmart Store Sales Forecast using Time Series Models

Primary Objective

Predict the extent of impact of holidays on sales of Walmart retail stores.

Overview

One challenge of modelling retail data is the need to make decisions based on limited history. If Christmas comes but once a year, so does the chance to see how strategic decisions impacted the bottom line.

The purpose of this analysis is to use an appropriate(limited) data from a retail store to predict the extent of impact of key holidays/events on the sales of a certain category of products.

Keeping the above objective in mind, the results can be evaluated using Weighted Mean Absolute Error (WMAE) where holiday week will have weight of 5 and rest will have a weight of 1.

Data Used

Here, we will use the historical sales data for 45 Walmart stores located in different regions. Each store contains many departments, and we will use time-series models to project the sales for one of the departments in each store. We will be using the data available on Kaggle.

Below 3 datasets will be used in our analysis:

- Stores.csv

  This file contains anonymized information about the 45 stores, indicating the type and size of store.

- Train.csv

  This is the historical training data, which covers to 2010-02-05 to 2012-11-01. Within this file you will find the following fields:
  Store -- the store number
  Dept -- the department number
  Date -- the week
  Weekly_Sales -- sales for the given department in the given store
  IsHoliday -- whether the week is a special holiday week

- Features.csv

  This file contains additional data related to the store, department, and regional activity for the given dates. It contains the following fields:
  Store -- the store number
  Date -- the week
  Temperature -- average temperature in the region
  Fuel_Price -- cost of fuel in the region
  MarkDown1-5 -- anonymized data related to promotional markdowns that Walmart is running. MarkDown data is only available after Nov 2011, and is not available for all stores all the time. Any missing value is marked with an NA.
  CPI -- the consumer price index
  Unemployment -- the unemployment rate
  IsHoliday -- whether the week is a special holiday week

Approach

The planned approach is to first have an idea of the available input variables and do any pre-processing required to prepare the time-series data. Next, we analyse the time-series data for any available patterns of seasonality, trend and autocorrelation. If there is a non-stationary series, we convert it to stationary and do the relevant statistical tests.

The algorithms we will be using for our purpose are time-series Decomposition, Exponential Smoothing, ARIMA (AR/MA etc as needed based on analysis), SARIMAX and Prophet.

Data Preparation

A quick analysis suggests that there are 2 types of stores, A & B, and multiple departments. We will be selecting only one department for our purpose, say department 4. We have missing info in features data in 'markdown' column which contains promos that we can ignore in our current analysis, same for CPI and unemployment rate. Now, we merge the required info in a single data and set date column as datetime index of weekly frequency.

We merge all the datasets and prepare a clean data with datetime index and all available features:

Date	Store	Dept	Weekly_Sales	IsHoliday	Type	Size	Temperature	Fuel_Price	CPI	Unemployment
2010-02-05	1	4	39954.04	False	A	151315	42.31	2.572	211.096358	8.106
2010-02-12	1	4	35351.21	True	A	151315	38.51	2.548	211.242170	8.106
2010-02-19	1	4	36826.95	False	A	151315	39.93	2.514	211.289143	8.106
2010-02-26	1	4	34660.16	False	A	151315	46.63	2.561	211.319643	8.106
2010-03-05	1	4	38086.19	False	A	151315	46.50	2.625	211.350143	8.106

As per our requirement, we pivot the dataset and arrive at a data with all time series columns that we need to predict. The index frequency is set to weekly and its split into train : test with ratio 80:20. Train data has >2 years of data and test has >6 months of data. Holidays list is also extracted and kept separately for our future use.

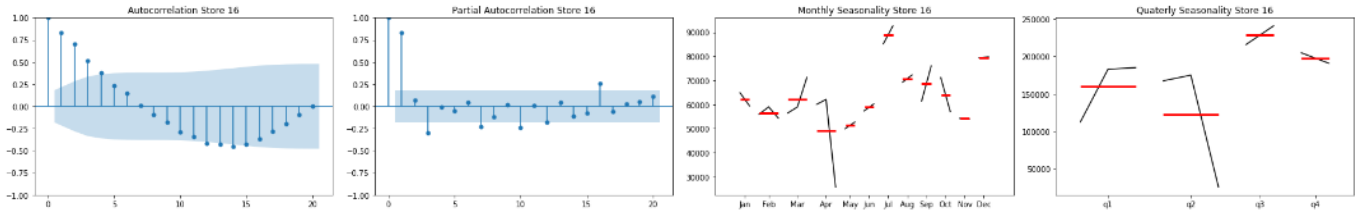
Store	1	2	3	4	5	6	7	8	9	10	...
2010-02-05	39954.04	47681.96	8495.10	59554.57	9282.86	35192.87	13418.73	21759.76	17483.90	51885.86	...
2010-02-12	35351.21	44197.95	8489.46	54069.82	9266.08	30612.26	12917.37	19921.07	15667.00	49359.18	...
2010-02-19	36826.95	46131.14	7824.13	53939.17	9595.35	31553.54	12230.31	20303.28	15069.46	50350.28	...
2010-02-26	34660.16	42126.71	8012.43	54687.08	9297.39	30353.02	12838.59	18821.94	15537.55	51206.84	...
2010-03-05	38086.19	46937.81	8511.73	56959.02	9610.15	35574.46	12439.00	21336.02	16630.68	50641.43	...

holiday	ds
SuperBowl	2010-02-12
SuperBowl	2011-02-11
SuperBowl	2012-02-10
LaborDay	2010-09-10
LaborDay	2011-09-09
LaborDay	2012-09-07
Thanksgiving	2010-11-26
Thanksgiving	2011-11-25
Christmas	2010-12-31
Christmas	2011-12-30

Time Series Exploration

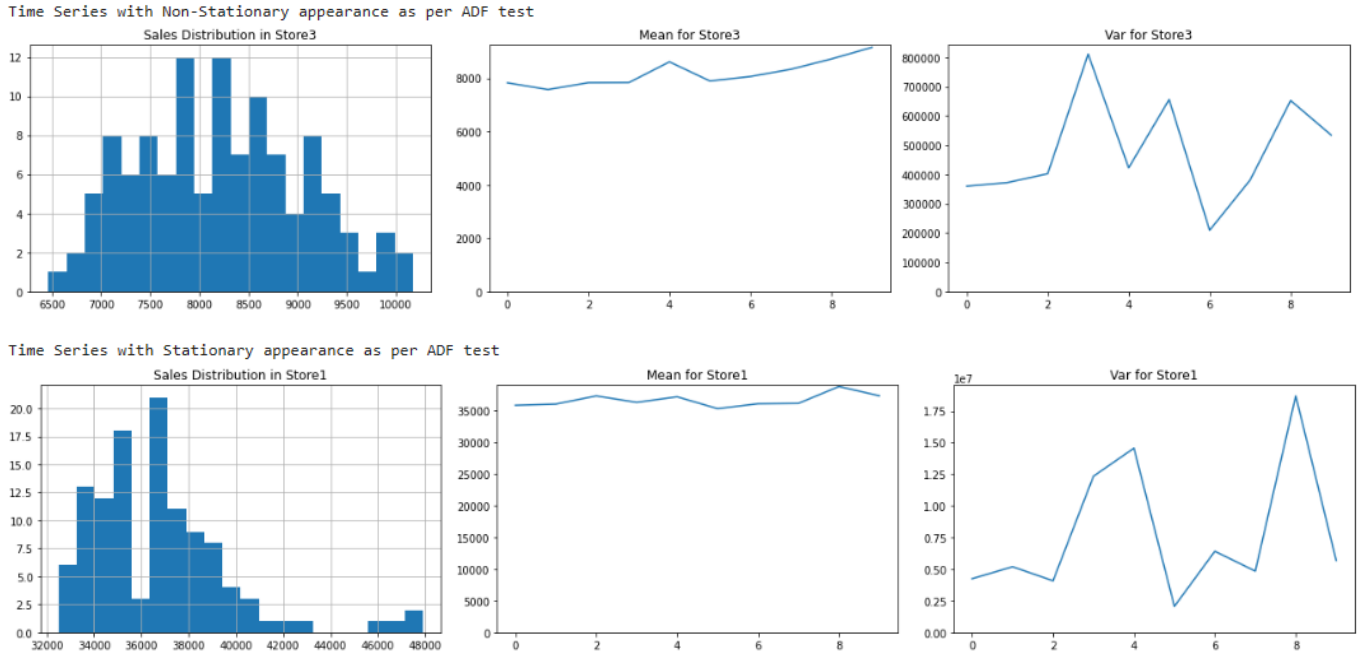
We plot all of 45 available time series of weekly frequency. We cannot make much out of this weekly plot. It shows that some of the stores have a pretty much stable mean and stddev and rest have high variations/mild trend so we might have multiple non-stationary time-series.

So, we further plot monthly/quarterly sales (total & smoothened plots using rolling mean). We look at acf, pacf, monthly & quarterly seasonality plots using statsmodels library. For instance, store 16 has below output



From these plots we conclude that April has outlier value for the third year which causes quarter 2 values to vary a lot in almost all of the stores. For some of the stores there seems to be a pattern of growing sales going from January to December.

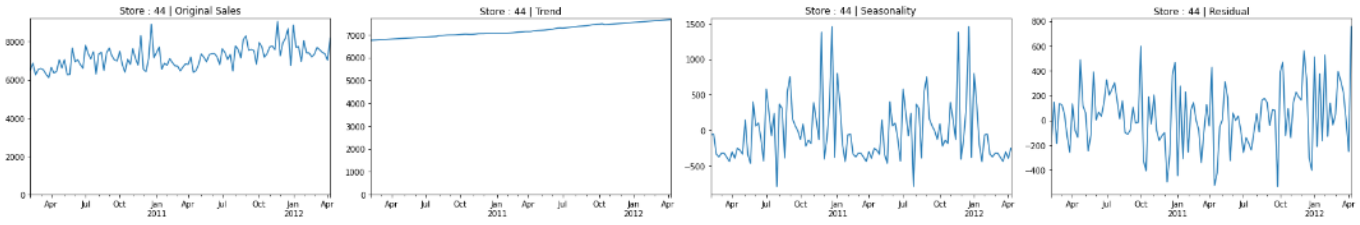
No consistent seasonality can be determined from above as some months/quarters seem to have a lot of variability. Also, a few stores seem to have non-constant autocorrelated values, hence must be non-stationary. Based on ADF test at p-value of 5%, half of the time series are non-stationary. Since heteroscedastic time-series can sometimes have misleading ADF test results we check the distribution of values in each non-stationary time series.



These plots lead us to conclude that the non-stationary time series have slight trend in some cases and heteroscedasticity in almost all the cases which could be due to the seasonality. For these stores, we can try additive seasonal decomposition method with annual seasonality.

The time-series that are stationary (as per ADF test) have a stable mean; pretty much normal distribution with some outliers (perhaps also resulting in unstable variance). In these cases, heteroscedasticity could be from these outliers or from possible weekly/monthly seasonality.

Additive Decomposition works better as compared to multiplicative as evident from the series plots. Below is an instance of aforementioned decomposition for a randomly selected store.



We can clearly see the trend and seasonality components. Also, plots suggest that even stationary series (based on ADF test) have trends. Our takeaway from this exercise is that almost every series has additive trend and annual seasonality. This should be helpful in applying triple smoothing & auto-arima algorithms in the following sections.

```
ADF Test Result on original series :
Non Stationary    23
Stationary        22
Name: stationary, dtype: int64

ADF Test Result on residual after removing trend and seasonality :
Stationary        30
Non Stationary    15
Name: stationary, dtype: int64
```

Time Series Model – Triple Exponential Smoothing

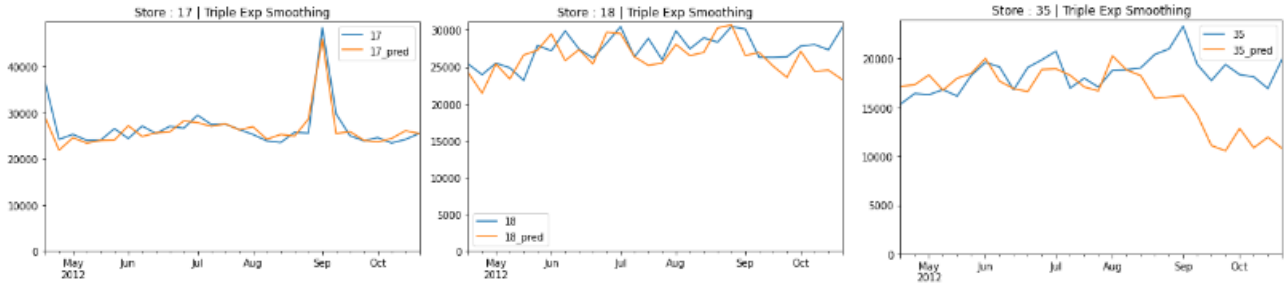
Before training any models, we create a function to calculate the Weighted Mean Absolute Error. So, we want a model with as low WMAE error as possible.

```
def weighted_mae(df, w, y, y_pred):
    '''
    This function calculates weighted mean absolute error.
    The input dataframe 'df' should have weights in column 'w', targets in column 'y' and predictions in column 'p'.
    '''

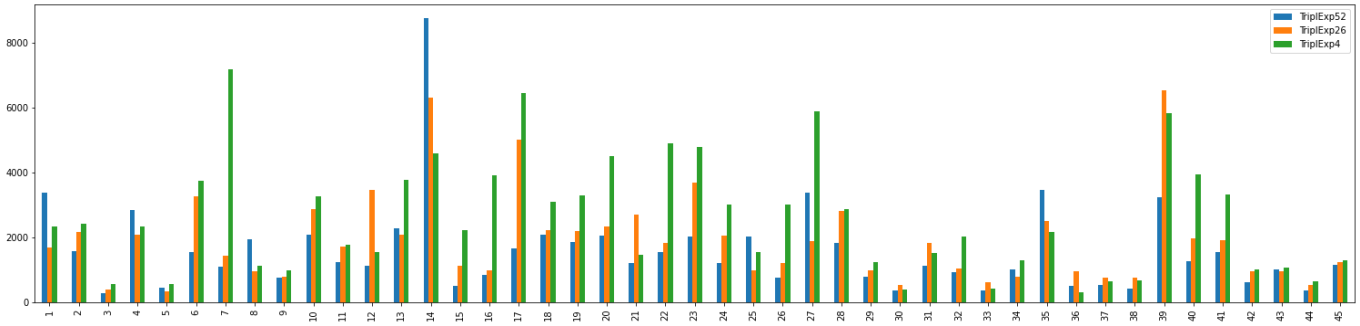
    return (abs(df[y]-df[y_pred])*df[w]).sum() / df[w].sum()
```

The first algorithm we attempt is Triple Exponential Smoothing because it can capture both trend and seasonality. A few experiments quickly give an indication of better predictability with monthly and even half-yearly seasonality.

Hence, using this algorithm we develop 3 models for each store with different seasonal_periods (52, 26, 4). In some cases it does a good job predicting the sales curve(along with occasional high) but not for all the stores as we can see below.



The 3 exponential smoothing models have only one difference and that is seasonality. It seems there are some stores which have always high errors. It's possible that these stores have high sale values as well. For the rest, most of the times monthly seasonality results in much higher error than the rest (especially, store 14 onwards). In other cases, all 3 are quite comparable (store 1-6, 8-14).



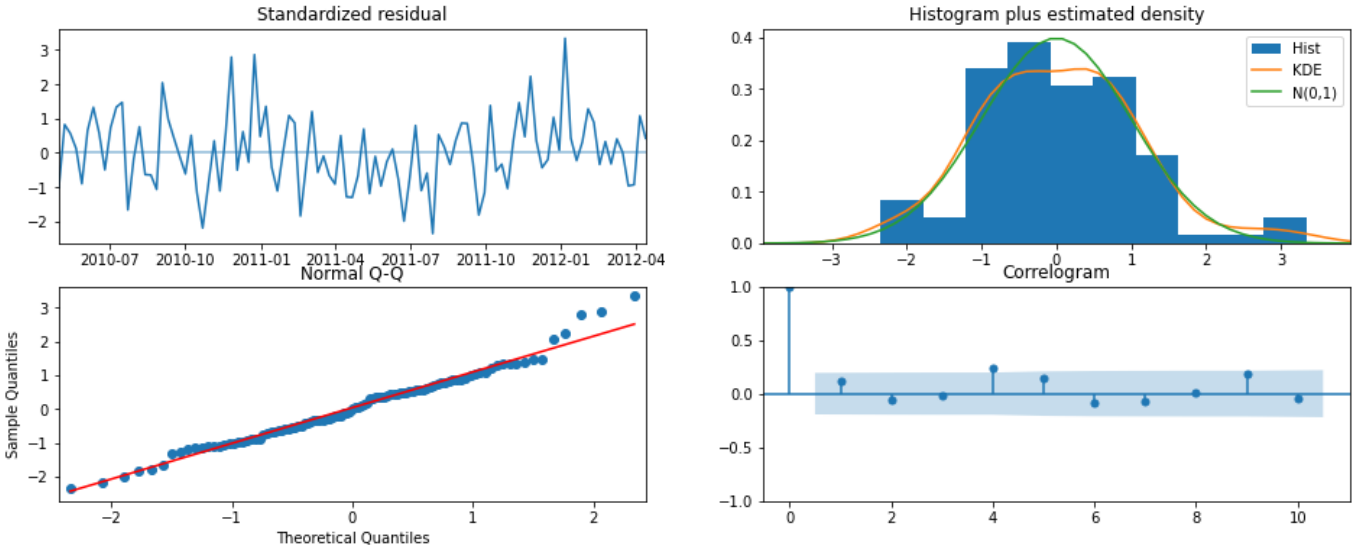
Time Series Model – SARIMA models using auto-arima

We use pmdarima library to obtain hyperparameters for SARIMA models for all the stores. This library speeds up the process but has a risk of over complicating the model so we keep the maximum model order to a default of 5.

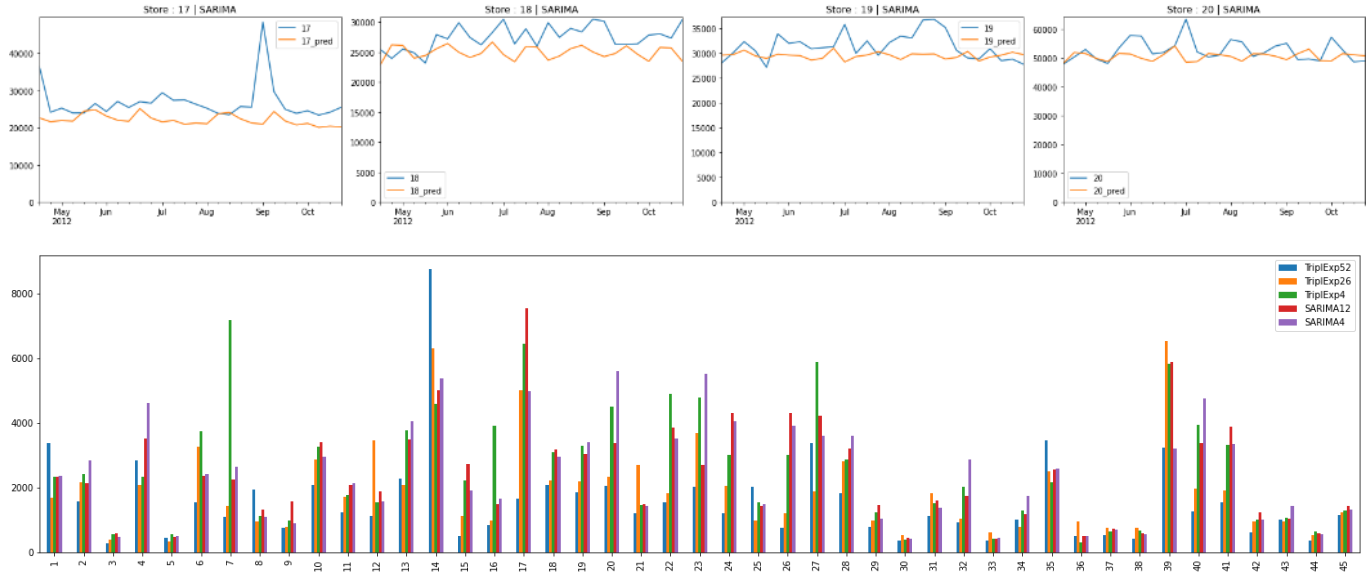
```
stepwise_model = pm.auto_arima(project_train[store_id],
                               start_p=1, start_q=1, max_p=3, max_q=3, d=0,
                               seasonal=True, m=12, start_P=0, D=1, trace=False,    # set trace to True to get stepwise aic scores and model config
                               error_action='ignore', suppress_warnings=True, stepwise=True)
```

Using the stepwise model search and experimenting a bit with above code suggests that m=12(monthly seasonality) and m=4(quarterly seasonality) are better than m=1(annual seasonality) which fails to capture any variation in the series.

Residuals are deviating from normality at extremes which could perhaps be fixed through log transformations. A suggestion to keep in mind for future.



The algorithm fails to capture the series pattern for multiple stores as seen below. This could be due to the constraints on model order or maybe because we didn't try to change P and Q for seasonal components by much.



The new SARIMA models we just added have very high errors. Perhaps log transform would have reduced variations by some degree or changing P and Q as mentioned above.

Time Series Model – Prophet

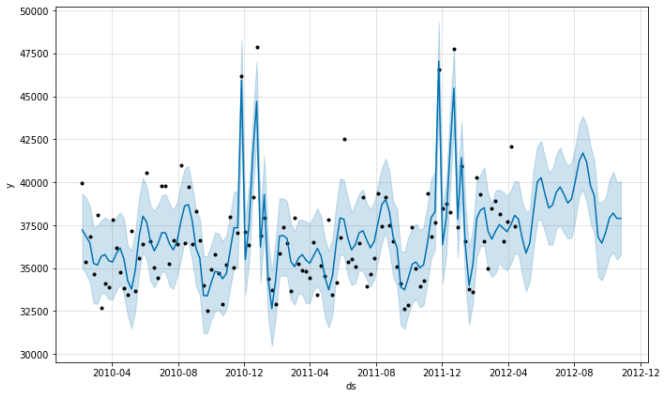
We use prophet library from Facebook to develop a more complex model hoping this will be able to capture the complicated patterns our previous models have missed. We train a sample model with default seasonality and another one with monthly seasonality included since we have suspected this to be present from the beginning.

```
store_id = 1

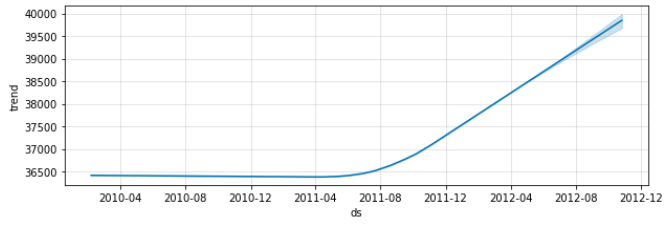
prophet_train = pd.DataFrame({'ds' : list(project_train.index), 'y': list(project_train[store_id])})
fbproft = Prophet(holidays=holidays)
fbproft.add_seasonality(name='monthly', period=30.5, fourier_order=5)   # Add in another seasonality besides yearly, weekly, daily
fbproft.fit(prophet_train)

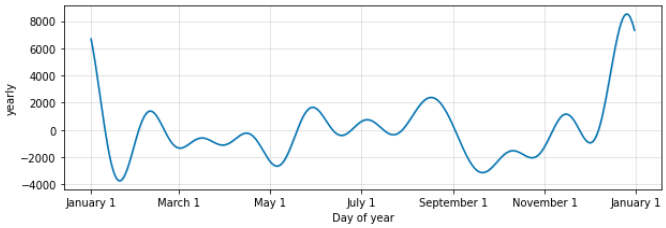
future = fbproft.make_future_dataframe(periods=len(project_test[store_id]), freq='W-FRI')   # freq can be any freq valid in pd.date_range
fbproft_pred = fbproft.predict(future)
```

We get some good results without monthly seasonality in sample run for store 1 but that is just for one store. The prophet model captures trend and seasonality and even impact of all the holidays as seen below.

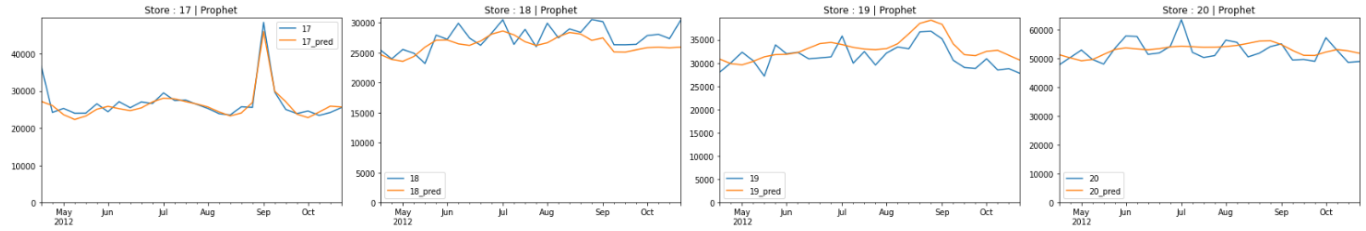


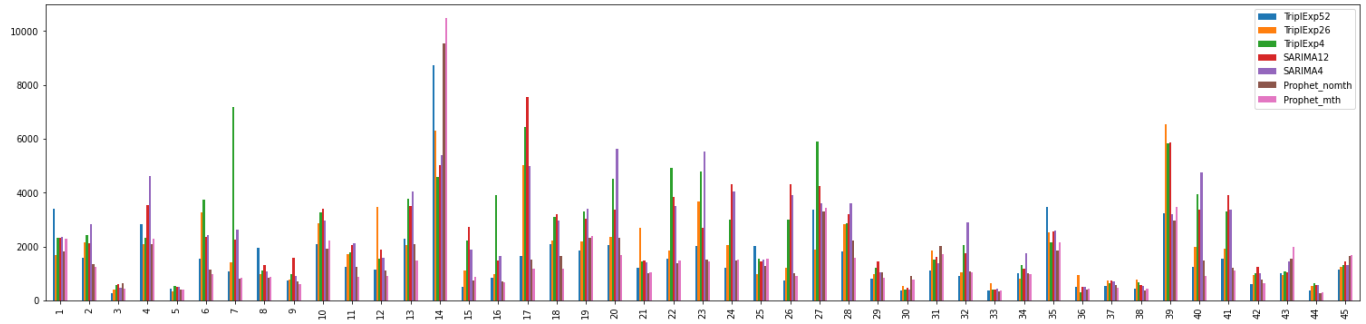
ds	holidays	Christmas	LaborDay	Thanksgiving	SuperBowl
2010-02-12	-864.046552	0.000000	0.000000	0.00000	-864.046552
2010-09-10	1104.077842	0.000000	1104.077842	0.00000	0.000000
2010-11-26	9859.635580	0.000000	0.000000	9859.63558	0.000000
2010-12-31	-7596.057851	-7596.057851	0.000000	0.00000	0.000000
2011-02-11	-864.046552	0.000000	0.000000	0.00000	-864.046552
2011-09-09	1104.077842	0.000000	1104.077842	0.00000	0.000000
2011-11-25	9859.635580	0.000000	0.000000	9859.63558	0.000000
2011-12-30	-7596.057851	-7596.057851	0.000000	0.00000	0.000000
2012-02-10	-864.046552	0.000000	0.000000	0.00000	-864.046552
2012-09-07	1104.077842	0.000000	1104.077842	0.00000	0.000000





Prophet without monthly seasonality has below sample outputs. This shows how some patterns are easily captured (store 17) while some are not(store 20). The store 20 might have frequent changes owing to monthly seasonality so it might help to train prophet with monthly seasonality for such series.





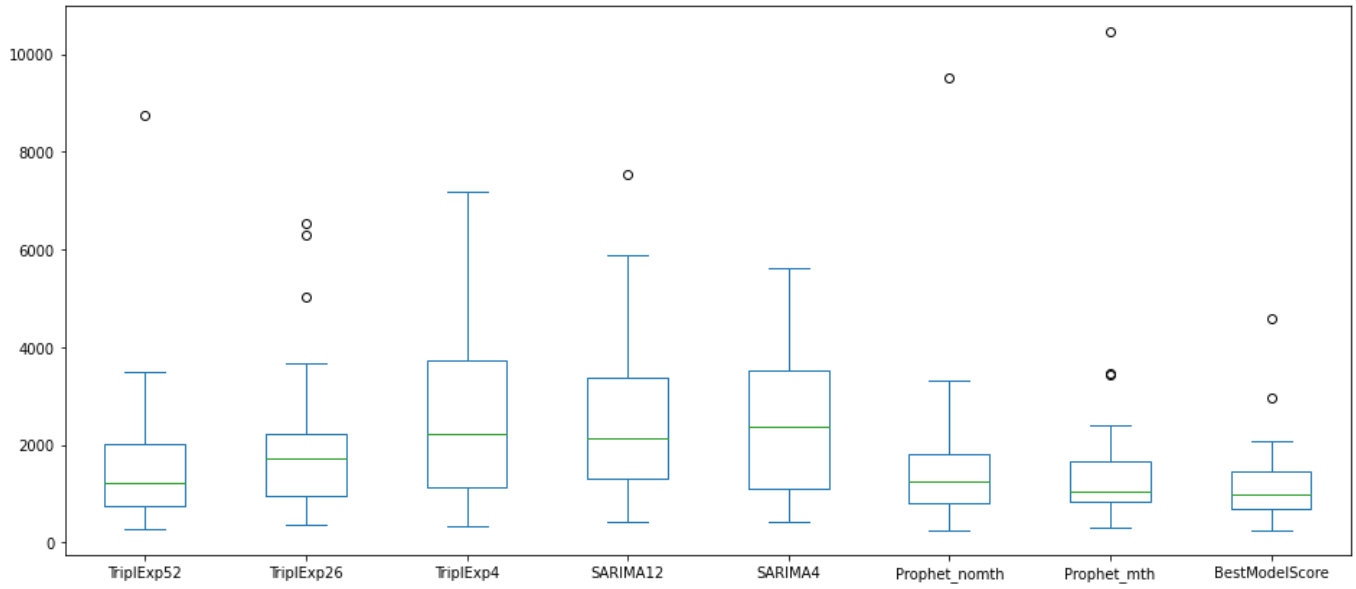
Now, we have all the models prepared to be compared. Except a few cases, Prophet models are either better than other models or comparable when looked at WMAE scores (or rather errors). This is to be expected because this is the only model that factors in holidays to model their effect separately.

Key Findings and Insights

We had ~2.75 years of sales data of 45 Walmart stores for several categories of products and we chose to develop models for department 4. The goal was to predict the sales, primarily the impact of holidays on sales. We set a weight of 5 on holiday weeks as opposed to weight of 1 for normal weeks to focus more on holiday weeks while predicting.

Based on WMAE scores for all the stores' sales we observed below:

- Median WMAE score across all the stores is minimum for 'Triple Exponential Smoothing' with annual seasonality and both the 'Prophet' models.
- Variation of WMAE score is least in 'Prophet' with monthly seasonality but both the Prophet models have some extreme outliers.
- 'SARIMA' models obtained with auto-arima and 'Triple Exponential Smoothing' model with monthly seasonality have a lot of variation in WMAE scores and also have a comparatively higher median score than the rest mentioned above.
- More than half of the stores have 'Prophet' as the best model. Second to that is 'Triple Exponential Smoothing' model with annual seasonality.
- Best model has a very low median and variation in WMAE scores. Only a few outlier stores have a score of 2000+, 95% of the stores have a score of less than 2000.



Our observations have led us to conclude below:

- We started with 45 stores' sales time series, each having its own distinct trend and seasonality pattern. Additive Decomposition showed us better improvement in stationarity and also gave an indication of how time-series components look like in each of them.
- We developed a total of 7 models for each of the 45 stores using 3 specific algorithms namely Triple Exponential Smoothing, SARIMA and Prophet. We also looked into the possibility of annual, weekly and monthly seasonality wherever possible.
- In the end, we compared WMAE(Weighted Mean Absolute Error) scores for all the models. This led us to the conclusion that out of 7 models Prophet stood out as the one with best score in more than half of the stores. The best model has a very good score(less error) as compared to what we even obtained with the auto-arima process.

BestModel	Count
Prophet_mth	15
TriplExp52	11
Prophet_nomth	10
TriplExp26	7
TriplExp4	2

Future Improvements

We can still make several improvements to each of the models trained above. Below are some of the ideas for improvement.

- Train Exponential Smoothing & auto-arima on log transform and/or train it on exogenous variables to improve result. Log transformation will deal with outliers and exogenous variables will add more info to train the models on.
- Use linear Regression on trend, Fourier components (coupled with periodograms) on seasonality & ARMA on stationary residuals to make the most of decomposition components.
- Prophet can be improved further with addition of more holidays, more independent features like unemployment, promo codes, fuel prices, weather etc present in the original input data.
- If GPU is available, try RNN/LSTM to see if this improves the predictability.

Useful Links

- Data Source : https://www.kaggle.com/competitions/walmart-recruiting-store-sales-forecasting/data
- Project Code : Python-QuickStart-Codes---Stats-ML-DL-AI-NLP/Time_Series_Models_for_Walmart_Sales Forecast.ipynb at main · quickSilverShanks/Python-QuickStart-Codes---Stats-ML-DL-AI-NLP (github.com)